TECHNICAL PROJECT REPORT

Inventory Management System

with DAO Pattern and File Persistence

Subject: Object-Oriented Programming with Java
Term: II | **Year:** 2024–25
Repository: rish0422/term-ii-project-submission-rish0422

1. Problem Statement

Small businesses and retail stores often struggle to manage their product inventory efficiently without access to expensive enterprise software. Fragmented manual tracking leads to stockouts, overstocking, and lost revenue — all of which could be prevented with the right tool.

This project addresses that gap by providing a lightweight desktop application to manage products, track stock levels, and raise alerts when items run low. The system uses the DAO (Data Access Object) design pattern to separate business logic from data persistence, storing all data locally in a binary file (products.dat) without requiring any external database or internet connection.

A clean Java Swing GUI enables non-technical users to perform all inventory operations intuitively — from adding new products to restocking and deducting quantities.

2. Target User

The primary users of this system are:

- Small business owners who need a simple, offline desktop tool
- Shopkeepers and retail store managers in tier-2 and tier-3 cities
- Any individual or small team that requires inventory tracking without the overhead of cloud-based ERP systems

The system is designed to be operable by users with no programming background, relying entirely on a GUI — no command-line interaction is required during normal usage.

3. Core Features

Product Management: Add, edit, and delete products with full field validation.

Stock Operations: Restock and deduct quantities with appropriate guard rails to prevent negative stock.

Search & Filter: Search products by name or category in real time.

Low Stock Alerts: Products below a defined threshold are highlighted in red in the product table, providing instant visual cues.

Data Persistence: All data is saved to a binary file (products.dat) and survives application restarts — no database setup required.

Intuitive GUI: A clean Java Swing interface with labelled panels, buttons, and input forms suited for non-technical users.

4. OOP Concepts Used

The following table summarises the six core OOP concepts applied in this project, the files where they are implemented, and how each concept contributes to the overall design.

OOP Concept	File	Description
Abstraction	ProductDAO.java	Interface defining all data operations without exposing implementation details
Inheritance	ProductDAOImpl.java	Implements ProductDAO interface, inheriting its contract with file-based logic
Polymorphism	ProductService.java	Holds a ProductDAO reference pointing to ProductDAOImpl at runtime
Encapsulation	Product.java	Private fields exposed only via public getters/setters; validation hidden in Service
Exception Handling	All layers	Errors caught at UI layer and shown via JOptionPane; prevents application crashes
Collections	ProductDAOImpl.java	List<Product> (ArrayList) with Java Streams for filtering and searching

4.1 Abstraction

ProductDAO.java is a Java interface that declares all data operations — add, update, delete, findById, findAll, search — without specifying how they are performed. The UI and Service layers interact exclusively with this abstraction, remaining completely unaware of the underlying file I/O implementation.

4.2 Inheritance

ProductDAOImpl.java implements the ProductDAO interface, inheriting its full contract and providing the concrete file-based behaviour. This follows the foundational OOP principle of programming to an interface rather than to an implementation.

4.3 Polymorphism

ProductService holds a field of type ProductDAO. At runtime, this field is assigned an instance of ProductDAOImpl. Because the reference type is the interface, the service is agnostic about the underlying implementation — the concrete class can be swapped (e.g., to a DatabaseDAOImpl) without touching the business logic layer.

4.4 Encapsulation

Product.java declares all fields (id, name, category, price, quantity, lowStockThreshold) as private, and exposes them only through clearly defined public getters and setters. ProductService similarly hides all validation logic internally, exposing only clean, validated methods to the UI layer.

4.5 Exception Handling

All DAO and Service methods declare throws Exception. Specific failure scenarios — product not found, insufficient stock, invalid input, file I/O failure — are caught at the UI layer and presented to the user via JOptionPane.showErrorDialog, preventing unhandled crashes.

4.6 Collections & Streams

The application uses List<Product> (backed by ArrayList) throughout all layers for storing, retrieving, and passing product data. Java Streams API is used inside ProductDAOImpl for filtering and search operations, terminating with Collectors.toList() to return typed result lists.

5. System Architecture

The application follows a strict 3-layer (MVC-inspired) architecture where each layer communicates only with the layer directly below it. This separation of concerns ensures maintainability, testability, and extensibility.

Layer	Components	Responsibility
UI Layer (Swing)	MainFrame, ProductTablePanel, ProductFormPanel, SearchPanel	User interaction, data display, input capture
Service Layer	ProductService	Business logic, input validation, orchestration
DAO Layer	ProductDAO (interface) → ProductDAOImpl	All file read/write operations and data access
Persistence	products.dat	Binary file storing serialized List<Product> objects

5.1 Data Flow

A typical user interaction follows this path:

1. User triggers an action (e.g., clicks 'Add Product') in the UI layer.
2. The UI panel collects input, validates it superficially (empty check), and calls the appropriate ProductService method.
3. ProductService performs deep validation (e.g., price > 0, quantity ≥ 0) and calls the appropriate ProductDAO method.
4. ProductDAOImpl reads the current list from products.dat, modifies it, and serialises the updated list back to the file.
5. Control returns up the call stack; the UI refreshes the product table.

5.2 File Persistence Design

The persistence layer serialises the entire List<Product> object to a binary file using Java's ObjectOutputStream and deserialises it on startup using ObjectInputStream. The Product class implements Serializable, allowing Java's native serialisation mechanism to handle binary encoding automatically.

This approach requires zero database configuration and works completely offline — the data file is created automatically on the first run and updated atomically on every write operation.

6. How to Run

6.1 Requirements

Java Version	Java 17 or higher
Supported OS	Windows, macOS, Linux (any OS with a JVM)
Database	None required — uses local binary file (products.dat)
Network	Fully offline — no internet connection required

6.2 Setup & Execution

Follow these steps to clone, compile, and run the application:

Step 1 — Clone the Repository

```
git clone https://github.com/rish0422/term-ii-project-submission-rish0422.git
cd term-ii-project-submission-rish0422
```

Step 2 — Compile All Source Files

```
javac src/*.java
```

Step 3 — Run the Application

```
java Main
```

The application window will open immediately. On first launch, products.dat is created automatically in the working directory. All subsequent launches will load existing product data from this file.

7. Project Structure

File / Directory	Purpose
`src/Product.java`	Entity class — encapsulates all product fields with getters/setters
`src/ProductDAO.java`	Interface — defines the data access contract (abstraction)
`src/ProductDAOImpl.java`	Implementation — file-based DAO with serialisation logic
`src/ProductService.java`	Service layer — business logic, validation, and DAO orchestration

File / Directory	Purpose
`src/MainFrame.java`	Main Swing JFrame — entry point and layout host
`src/ProductTablePanel.java`	Displays product list with low-stock red highlighting
`src/ProductFormPanel.java`	Form for adding and editing products
`src/SearchPanel.java`	Search bar and filter controls
`Main.java`	Application entry point — instantiates MainFrame
`products.dat`	Auto-generated binary persistence file (created at runtime)

8. Conclusion

This project demonstrates the practical application of core Object-Oriented Programming principles in a real-world desktop application scenario. By implementing the DAO pattern, the system achieves a clean separation of concerns across three distinct architectural layers — UI, Service, and DAO — each of which can be independently modified or replaced.

The use of Java's built-in serialisation for file persistence eliminates the need for any external dependencies or database setup, making the application truly portable and deployable on any standard Java environment. The Java Swing GUI ensures accessibility for non-technical end users, while the underlying design remains extensible enough to support future enhancements such as database-backed persistence, CSV export, or a REST API layer — without requiring changes to the business logic.

In summary, the Inventory Management System successfully bridges the gap between technical design rigor and practical usability for small business environments.

— End of Report —